Production Deployment

From working to operated: serving, containers, secrets, limits, scheduling, rollback, the Heavy-lane gates, and the release ceremony

SURVEY | BUILD | GOVERN | RELEASE

CIDM 6333 · Agentic Systems Design and Development
Survey: background and cloud agents · gzkit closeout, patch release, MX · Build: release and rollback in one governed command
Code freeze and closeout ceremonies: dates in the course calendar

Two hours, seven moves

The run of show

1	0:02	**Framing**	From working to operated; the release is a thing; secrets are injected, never stored
2	0:10	**Spine I**	Serving an agent (service or job); a FastAPI excerpt; containers, secrets, limits; two ways to ship
3	0:33	**Spine II**	Unattended runs; Nygard's stability patterns for agents; Heavy-lane gates; the common mistake; the runbook
·	0:55	**Break**	Five minutes
4	1:00	**Teardowns**	Background agents compared; Copilot cloud agent; scheduled agents; gzkit closeout; live Design Analysis
5	1:30	**Reference design**	Distribution invariant and backfilled releases; the 5× over-count; V.I.B.E.S.; MX mode
6	1:45	**Lens, card, close**	Evaluation lens; Flight Card 12; ceremony prep and booking; lab brief; the Sprint

From working to operated

Working means it ran for you. Operated means someone else can release it, see it, and take it back.

A release you have never rolled back is a release you cannot vouch for.

Two rules the spine returns to

The release is a thing, not a moment: an artifact with a version and a hash. Rollback reinstalls it; no rebuild.

Secrets are injected at run time. Never in the image, the log, the repository, or the model's context.

Tonight's claim, which Sortie 12 will try to disprove

A release rolls back in one governed command; the ledger names the released hash; a planted canary secret appears in no image, log, or commit.

Falsifiers: a manual step; a hash mismatch; the canary found; no attestation event.

Serving an agent: a service or a job

Decide which one you are shipping before you choose a framework

Service (request/response)

A caller sends a request and waits. FastAPI fronts your loop.

Each request has a time budget; the loop halts inside it

A retried POST must not run twice: idempotency key

Long answers stream

/health reports status and version

Job (run to completion)

A trigger starts a run; nobody waits on a socket.

Budget is turns and dollars, not a request timeout

Output is an artifact: branch, report, or ledger entry

The result must say what it did

Exit code is the only status the scheduler sees

Choose, and write it down

Most course systems are jobs wrapped as a service for the demo. Pick one in an ADR; state the lifecycle.

The mistake: a service with job semantics. A ten-minute POST; a client that times out and retries.

A request that outlives its caller becomes an orphan. Decide who owns the halt: loop budget or server timeout.

A served agent in thirty lines (FastAPI)

Run with: fastapi run src/myagent/serve.py (fastapi[standard]; production server, 0.0.0.0:8000 by default)

```
import asyncio, os                        # src/myagent/serve.py; agent_loop() is your E1-E11 loop
from contextlib import asynccontextmanager
from importlib.metadata import version
from fastapi import FastAPI, Header, HTTPException

TURN_TIMEOUT_S = float(os.environ.get("TURN_TIMEOUT_S", "30"))   # limits from config
_seen: dict[str, dict] = {}              # idempotency store; in-process (restart forgets)

@asynccontextmanager
async def lifespan(app):
    app.state.api_key = os.environ["MODEL_API_KEY"]   # injected; missing = fail at boot
    yield                                             # shutdown: flush ledger, close clients

app = FastAPI(lifespan=lifespan)

@app.get("/health")
def health():
    return {"status": "ok", "version": version("myagent")}   # rollback visible here

@app.post("/runs")
async def run(task: dict, idempotency_key: str = Header(...)):   # Idempotency-Key
    if idempotency_key in _seen:                  # a retried POST gets the first result
        return _seen[idempotency_key]
    try:
        result = await asyncio.wait_for(agent_loop(task), TURN_TIMEOUT_S)
    except TimeoutError:                          # halt inside the caller's budget
        raise HTTPException(504, "agent exceeded its time budget; halted")
    _seen[idempotency_key] = result
    return result
```

Containers, secrets, limits

Image, config, secret, limit, log: each has a wrong place and a check the Chase can run

Concern	Wrong place	Right place	The Chase's check
Image / artifact	Built from a dirty tree; tagged latest; rebuilt for rollback	Built from a clean commit; version and hash recorded at release	Hash in the release event equals the hash of what is running
Config	Constants in code; a limit stated in a Markdown file	Environment or one config file, read at startup and logged (values, not secrets)	Change the limit without a rebuild; /health or the log shows it
Secrets	ENV or COPY in a Dockerfile; .env committed; key echoed in a trace	Injected at run: docker run --env-file (untracked), OS keychain, CI secret store	grep the image history, logs, and git log for the canary: zero hits
Rate limits and budgets	The provider's limit is the only limit	Per-run turn and dollar budget; per-key requests per minute; a 429 with Retry-After	Exceed the budget on purpose; the run halts and the ledger says why
Logs and traces	Full prompts with user data; tool arguments that carry the key	Structured events; redaction before write; the key never enters the model's context	Search the trace store for the canary and for a real user field

FastAPI's container guidance: one process per container; exec form of CMD; --proxy-headers behind a TLS proxy (fastapi.tiangolo.com/deployment/docker).

A secret in any image layer is in every copy forever. Deleting it in a later layer does not remove it.

Two ways to ship the same release

Tonight's lab offers both; the Sprint asks for one of them to be flown end to end

No container: a versioned wheel

uv version --bump patch → pyproject says 0.1.1
uv build → dist/myagent-0.1.1-py3-none-any.whl
Install into a deploy environment: uv venv deploy-env, then uv pip install the wheel.
Rollback: reinstall the prior wheel from the release record; check its SHA-256 first.
Works on every laptop; no daemon.

Container: an image

docker build -t myagent:0.1.1 . (or podman build)

The tag is a pointer; the image ID or digest is the release. Record the ID in the release event.

Rollback: start the prior image by ID.

Needs Docker or Podman; the lab does not require it.

A version number is a name. The hash is the release. Record it at release; check it at rollback.

Unattended runs: who attests a job nobody watched?

Scheduling and background agents (21.3): the five controls every vendor now ships in some form

1 → 2 → 3 → 4 → 5

Stored trigger

The prompt and schedule are written ahead of time by an authorized person. The firing proves storage, not consent.

Isolated environment

Fresh VM or runner per run; network limited by default; secrets held outside the sandbox.

Narrow write path

One branch with a known prefix (copilot/, claude/, codex/). No push to a protected branch; no agent merge.

Human merge

A person with write access reviews the diff and merges. CI on the agent's branch may wait for approval.

Record and read

A run log exists. Green means the run exited, not that the task succeeded. Somebody reads the transcript.

Nobody watched the run. Its record must answer: what changed, under whose authority, who accepted it.

Nygard's stability patterns, applied to an agent

Release It!, 2nd ed. (Pragmatic Bookshelf, 2018), 'Stability Antipatterns' and 'Stability Patterns': six that fit a loop with tools

Timeouts

Every model call and tool call has one. Do not retry at once after a timeout. Queue or halt.

Circuit Breaker

After N failures, stop calling the provider or tool and fail the run with a reason. Breaker state in /health.

Steady State

Everything that accumulates must be purged by something: traces, the idempotency store, memory rows, old wheels. No manual clean-up.

Fail Fast

Check key, model route, and budget before the first turn. A run that cannot finish should not start.

Governor

Slow automation down in the unsafe direction so a human can intervene: deletes, deploys, mass writes need a person.

Force Multiplier (antipattern)

Automation that amplifies one wrong decision across the fleet. A scheduled background agent multiplies whatever its prompt gets wrong.

The Governor is the course's pattern since E3: a write needs a confirmation event; an irreversible act needs a human.

The Heavy lane and the release ceremony

Lite ran gates 1–2 all term. The Final System runs 1–5. Gates 3–5 are the ones a release adds.

Gate 3 · Docs

The README, runbook, and release notes describe the release that exists. A pasted command that fails is a gate failure.

Gate 4 · BDD

Scenarios for the external contract: what a caller of /runs or the CLI can rely on. Executed, not just written.

Gate 5 · Human

A person attests: Completed, Completed — Partial, or Dropped. The attestation is a ledger event. The agent never records it.

Release notes vs. changelog

The changelog lists every change, for maintainers. Release notes tell a user how to upgrade and roll back.

Chapter 21.5: docs gate; BDD for external contracts; human attestation; release notes vs. changelog; the distribution invariant; MX mode.

The common mistake: rollback by rebuild

Same package, same version number, two builds, run on 09/24 while writing tonight's lab

21cb6c3d…

relcheck 0.1.0 wheel, first build: SHA-256 recorded in the release event

4aefd5f6…

relcheck 0.1.0 wheel, rebuilt after the version was set back: same file name, different bytes

0.1 s

time to roll back by reinstalling the recorded wheel into the deploy environment

What happened. The version was set back to 0.1.0 and uv build ran again. The new wheel overwrote the old; the rollback installed it with a mismatched hash.

Why it passes a careless check. Version, file name, and health say 0.1.0. Only the hash differs.

Keep released artifacts where a build cannot overwrite them; make rollback refuse a hash that differs from the release event.

Worked example: the runbook is one page

The Sprint asks for docs/runbook.md. Someone who has never seen your system uses it at 2 a.m.

Section	What it says	How the Chase tests it
What runs where	Artifact type, where it runs, which port, which environment variable names	Start it from a clean shell with only these steps
Release	The one command, what it records, where the hash goes	Run it; find the release event and the hash
Health	The route or command; the expected output, with the version	Compare output with the release event
Rollback	The one command; the time you measured; what it refuses to do	Run it after a bad release; time it; compare hashes
Limits and budgets	Per-run turns and dollars; request rate; what happens when exceeded	Exceed one on purpose
Secrets	Where each secret comes from at run time; how to rotate it; the canary check	Run the canary search
Known gaps	What the runbook does not cover (e.g. in-memory idempotency store lost on restart)	Check the gap is real and stated

Break · 0:55–1:00

Five minutes

Question to bring back: your system ran something unwatched last week. Which record tells a stranger what it changed and who accepted it?

1:00–1:07

Teardown: background and cloud agents, September 2026

Read for runtime and governance: where it runs, what starts it, what it can write, who accepts the work

System	Where it runs	What starts it	Network by default	What it can write · who merges
Codex cloud (OpenAI)	Cloud container per task; setup script, then agent phase	Web/IDE task; @codex on an issue or PR; scheduled tasks in the app	Blocked in the agent phase; setup has access	A diff or PR; you apply or merge
Copilot cloud agent (GitHub; was 'coding agent' until 04/2026)	Ephemeral GitHub Actions environment; 59-minute session cap	Issue assignment, agents panel, @copilot on a PR, VS Code	Restricted by a firewall	Its own copilot/ branch; draft PR; cannot approve or merge
Claude Code cloud sessions	Isolated Anthropic-managed VM (or self-hosted)	claude.ai/code, mobile, desktop, claude --cloud	Limited by default (Trusted allowlist); can be disabled	A branch and PR you create; auto-fix pushes to its PR
Claude Code routines	A cloud session per run; research preview	Schedule (≥ 1 hour), API POST, GitHub PR or release events	Environment's setting (Default: Trusted)	claude/ branches; runs as your GitHub identity
Claude Code GitHub Action	A GitHub Actions runner in your repo	@claude mention, any workflow event, cron	The runner's network	What the workflow permissions allow; secrets from Actions

Checked 09/24/2026 against vendor docs (URLs in the final slide's notes). These products change monthly; re-check before class.

Teardown: Copilot cloud agent, read for mechanism

GitHub's PR-native background agent: the review is the gate

Mechanism

A task arrives from an issue, a panel, or a PR comment. The agent works in an ephemeral Actions environment prepared by .github/workflows/copilot-setup-steps.yml. It pushes to one copilot/ branch and opens a draft PR.

Design choice

Put the human gate where developers already look: the pull request. Only write-access users can start it. It cannot mark its PR ready, approve, or merge. Workflows on its PR wait for a person.

Cost

Actions minutes plus AI credits per session; a 59-minute ceiling. Review load moves to the humans: every task ends in a PR someone must read.

Failure mode

#9 Metagaming the gate: a PR that passes the repo's checks without doing the task. #1 if a workflow can skip a check. The defense: a reviewer who reads the diff.

Commits are authored by Copilot, co-authored by the assigning developer, and signed. Attribution answers who asked, not who checked.

Teardown: scheduled agents and the stored prompt

Claude Code routines, GitHub Actions cron, Codex scheduled tasks: a job that fires with nobody present

What the documentation says, and why it matters

Routines. The trigger 'attests only that the prompt was stored ahead of time' and 'can't act as approval or consent for actions during the run.'

Actions cron. Default branch; disabled after 60 idle days (public repos).

Codex scheduled tasks. Results: a Scheduled inbox.

Read for failure mode

#7 Hallucinated authorization: the schedule is not consent. Deny it in code.

#3 Fabrication by status: 'green' reported as 'done.'

Force Multiplier: one wrong prompt, repeated every night.

Governor: the run writes only to its branch; a person accepts; the ledger shows both events.

A schedule proves that someone wrote the prompt. It does not prove that anyone approved tonight's actions.

Teardown: gzkit closeout, patch release, and MX

A release is a ceremony with a record.

gz closeout --ceremony

Closes an ADR. The agent presents paths and commands, not conclusions; the human executes. The 11 steps live in src/gzkit/commands/ceremony_steps.py. Non-TTY: gz closeout <ADR> --ceremony --attest.

gz patch release

Cuts a patch release from closed work. Seventy-six releases, January to August 2026. Its 04-18 incident is on the reference-design slides.

gz validate --distribution

Checks the distribution invariant (ADR-0.0.31) before a release ships: what is distributed matches what the repository declares.

MX mode (ADR-0.0.74)

Maintenance mode after the June 2026 breakdown: 'loose in the bay, hard at the door.' Work in progress may be untidy; the door holds every gate.

Design Analysis prompt: which of these four would your system need first for a second person to release it?

Live Design Analysis on the board: Copilot cloud agent (2025–26)

Nine components, one line each; the class fills the 'why' and the judgment loop aloud

Component	Answer (from GitHub Docs and changelog, checked 09/24/2026)
Foundation	Hosted models GitHub selects or the user picks per task
Perception	Issue text, PR comments, repository files, instruction files (AGENTS.md, .github/copilot-instructions.md), configured MCP servers
Planning & reasoning	Research, plan, then code; since 04/2026 can work on a branch before a PR
Tools & orchestration	Shell, tests, linters inside the Actions environment; environment prepared by copilot-setup-steps.yml
Memory & context	The repository and its instruction files; only the branch and PR survive the session
Coordination	One agent, one repository, one branch per task; hands off through the PR; third-party agents assignable
Evaluation & feedback	Runs the repo's checks; reviewers comment with @copilot; CI waits for human approval
Governance & human interface	Write-access users only; cannot approve or merge; commits co-authored and signed; rulesets and admin disable
Runtime & operations	Ephemeral Actions runner; firewall; 59-minute session cap; billed in Actions minutes and AI credits

Judgment loop, aloud. Decision: the PR is the gate. Privileged: reviewability. Cost: review load. Failure mode: #9. Preservation: the agent cannot merge.

Reference design: the distribution invariant and the backfilled releases

What ships must be what was declared, and a release that is not recorded did not happen

ADR-0.0.31 and gz validate --distribution

The distribution invariant is an ADR; a validator checks it before release. What ships is what the repository declares.

For your system: the release event names version and hash; health reports the same version.

GHI #830: backfilled releases

Issue #830: releases recorded after the fact. The work shipped before the record existed.

A backfilled record proves what the tag holds now, not what users installed then.

Your release event written late is a squawk, not a pass.

Two rules: a validator checks the release before it leaves, and the record is written when the release happens, not later.

Reference design: counting releases and reopening main

Two dated incidents from Appendix G, both about a release record that looked right

5×

2026-04-18: gz patch release over-counted closed issues five times

8×

2026-05-30 → 06-08: green main reopened eight times in nine days

76

releases January–August 2026 (Preface)

04-18. The patch-release command over-reported closed work. The rule: 'Count what we CLOSE, not what we book.'

05-30 → 06-08. Green main kept being reopened. The failure class: V.I.B.E.S. Mechanisms: a pre-push gate, the MX hangar, MX mode.

For Sortie 12: release notes list what the ledger shows closed; a green check before push is a gate, not a courtesy.

Reference design: MX mode

2026-06-20, the 'utter shambles' audit, and ADR-0.0.74

Loose in the bay, hard at the door.

2026-06-20 audit (Appendix G): 70 validate scopes, one 162-parameter function, 116 handoffs. Response: MX mode.

Hangar: untidy and fast. Door (push, release, ceremony): every gate holds.

Your deploy environment is the door: rollback, canary search, attestation run there.

Evaluation lens: a strong Sortie 12

The Chase reads the release ledger, the hashes, the canary search, and the runbook, and nothing else

Weak

- Rollback is 'git checkout v0.1.0 and rebuild'
- Release event has a version but no hash
- Image tagged latest; no other record
- API key in a Dockerfile ENV, .env, or trace
- Runbook from memory; a pasted command fails
- Green status reported as success

Sound

- One command releases; one rolls back; both in the ledger
- Rollback event names the released hash
- Health reports the version, before and after
- Canary planted; image, logs, git log: zero hits
- Rollback timed; number in the runbook
- Human attestation event for the release

Exceptional

- Rollback refuses a mismatched hash
- Artifacts kept where a build cannot overwrite them
- Both paths flown, same ledger format
- Governor: background runs write to their branch; a person accepts
- Circuit breaker on the model route, in /health
- The Chase runs the runbook cold

Flight Card 12 brief — Sortie 12: a release that can be rolled back

Expansion · depends on Sortie 11 PASS · Go signed by your Chase in the repo by the date in the course calendar

```
ENVELOPE            expansion
DESIGN CLAIM        A release can be rolled back in one governed command; the rollback is in the ledger and
                    names the released hash; a planted canary secret appears in no image, log, or commit.
SYSTEMS UNDER TEST  serve entry (FastAPI or CLI) · build (uv build or image) · deploy env · release/rollback
                    command · release ledger · runbook · secret injection · attestation
SUBSTRATE           repo + commit · Python 3.14 · uv · model/route · Docker/Podman or none (state which)
DEPENDS ON          Sortie 11 PASS (tracing, cost table, guardrails)

TEST POINTS
  1  release N                    -> ledger {release, N, hash_N, ok}; health reports N
  2  release N+1 (planted defect)  -> smoke fails; ledger {release, N+1, ok:false}
  3  rollback (one command, timed) -> ledger {rollback, N, hash == hash_N, ok}; health reports N
  4  rebuild N over its own path, then rollback -> refused, or hash mismatch recorded (not ok)
  5  canary search: image history, logs, traces, git log -> zero hits
  6  attest the release           -> human attestation event in the ledger

INSTRUMENTATION     release ledger (>= 5 events) · hashes/IDs · health output x3 · rollback seconds · canary grep

PRE-REGISTERED FALSIFIERS (any ONE => FAIL)
  - rollback needs a manual step, or restores a hash different from hash_N
  - canary found in any artifact
  - a release with no attestation event
  - passed only by editing the ledger, skipping the smoke check, or deleting the defect

ABORT (halt-not-fail)  container engine unavailable (fly the wheel path instead); model route down
CHASE               reads the ledger lines, the hashes, the canary output, and runs the runbook's rollback cold

SPRINT (code freeze: see course calendar)  Sortie 12 log · served + packaged/containerized + release
                    + rollback · runbook · Design Analysis · AI_LOG   ·   ceremony booked
```

The closeout ceremony: what to prepare this week

Chapter 26 · 30–40 minutes by appointment · book on Calendly tonight · you execute, I observe

Element	What you bring or do	Evidence on the screen
Readiness	Clean tree at the freeze commit; gates 1–5 run on the Heavy lane	gz status; the gate output
Bill of materials	PRD, ADRs, briefs, code and tests, eval suite with red-team card, runbook, release, Design Analysis Ledger (13 rows), AI_LOG.md	Paths, opened one by one
Docs alignment	The README and runbook match what runs	A pasted command that works
Value walkthrough	What your system does for its user, in your words, without the agent	Your personalized brief beside it
Live demo	You run it; you release and roll back; I watch	The ledger lines as they appear
Attestation and audit	You attest; the record is checked afterwards	The attestation event; the audit output

Present paths and commands, not conclusions. If you catch yourself saying 'it works', open the file that shows it.

Lab tonight: release, break, roll back

Handout: CIDM6333_Lab_E12_Rollback.md · self-guided · no Docker required · the full card flies in the Sprint

Thirty minutes, six steps

0–4 Scratch package: uv init --package relcheck; add health().
4–10 Release 0.1.0: uv build; python deploy.py release <wheel>. Ledger line 1.
10–15 Break it: uv version --bump patch; plant a defect; build; release 0.1.1. Ledger line 2.
15–19 Roll back: python deploy.py rollback. Time it. Ledger line 3; compare hashes.
19–25 OpenCode as adversary: could the rollback succeed and still be wrong?
25–30 Container path if Docker or Podman works; else write the finding. Commit.

What you commit

evolution-12/lab/release-ledger.jsonl (3+ lines, including ok:false and a rollback)

evolution-12/lab/deploy.py

evolution-12/lab/worksheet.md: rollback seconds, both hashes, the model's claim and your verdict

A ledger event in your system recording the commit.

Tonight is the rehearsal for the Sortie 12 ledger format.

Small and free models name failure cases that do not exist. Test each claim against the script; a refuted claim is a result.

Homework: the E12 Sprint (closes at code freeze)

Brief: CIDM6333_HW_E12_Deployment-Closeout.md · everything committed by code freeze

Deliverables

- Sortie 12 flight log
- Served (FastAPI or CLI job) + packaged or containerized
- Release + one-command rollback, both in the ledger
- docs/runbook.md (seven sections)
- Design Analysis (a background agent or gzkit release) + LEDGER row
- AI_LOG entry

Asynchronous Go/No-Go and Chase

Your Chase reads evolution-12/flight-card.md and commits GO or NO-GO under it.

Fly by the date in the course calendar.

At code freeze the Chase runs your rollback from the runbook and writes a one-line PASS/FAIL.

Closeout prep

Book a Calendly slot for your closeout ceremony tonight.

The Final System, release, Design Analysis Ledger (13), and AI_LOG.md with final reflection are due on the date in the course calendar.

No code changes after the freeze; the ceremony runs the freeze commit.

The E12 Sprint closes at code freeze because the ceremonies start the next day.

You have not released it until you have rolled it back.

Record the hash when you release; check it when you roll back; keep secrets out of everything you ship. Card signed, code frozen, ceremony booked.